

PARTE 0 · TU ENTORNO DE PRÁCTICAS

De cero absoluto a nivel experto



BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Contenido de esta parte

Antes de empezar

0.1 Qué vas a necesitar

0.2 Instalar PostgreSQL

0.3 Abrir `psql` y conectarte

0.4 Tu primera orden: cómo se escribe una sentencia

0.5 Crear tus bases de datos de práctica

0.6 Escribir código en archivos y ejecutarlo

0.7 Herramientas gráficas (opcional)

0.8 Qué hacer cuando algo falla

0.9 Cómo practicar con este manual

Resumen de la Parte 0

Glosario de la Parte 0

Mini examen de la Parte 0

Parte 0. Tu entorno de prácticas

Antes de empezar

Aprender bases de datos leyendo no basta: se aprende **escribiendo código y viendo qué ocurre**. Esta parte te enseña, desde cero y paso a paso, a preparar tu ordenador para practicar: instalar PostgreSQL, abrir el programa donde se escribe el código, hacer tu primera orden, guardar tu código en archivos y qué hacer cuando algo falla.

No necesitas saber nada de antemano. Si nunca has abierto una «terminal», aquí se explica qué es.

Al terminarla tendrás:

- PostgreSQL instalado y funcionando en tu ordenador;
- el programa `psql` abierto y conectado;
- dos bases de datos de práctica creadas (`ensayo` y `tienda`);
- tu primer script (un archivo de código) escrito y ejecutado;
- un método para practicar el resto del manual y para resolver los errores más comunes.

Cómo se organiza cada punto

Cada punto sigue los pasos de siempre: **¿qué es?**, **¿para qué sirve?**, **¿por qué lo necesito?**, **¿cómo funciona?**, **paso a paso**, **código y resultado** (con captura real), ejemplos, **profundizando**, ejercicio y solución, error habitual, buena práctica y comprobación.

Qué está verificado y qué no

Las capturas de este manual se han generado ejecutando PostgreSQL 16.14 en **Linux**. Los pasos de instalación en **Windows**, **macOS** y **Docker** siguen los instaladores y la documentación oficial, pero **no he podido ejecutarlos en este entorno**, así que no hay capturas de esas pantallas. Si alguna pantalla de un instalador difiere de lo descrito, sigue la documentación oficial en postgresql.org/download. Lo que escribas dentro de `psql` funciona igual en todos los sistemas.

0.1 Qué vas a necesitar

¿Qué es?

Es la lista de piezas que hay que tener antes de empezar a practicar. Son pocas:

Pieza	Para qué sirve	Ejemplo
Un ordenador	Donde se instala y ejecuta todo	Windows, macOS o Linux
PostgreSQL	El gestor de bases de datos (el programa que guarda y consulta los datos)	Versión 16 o posterior
<code>psql</code>	El cliente de terminal : el programa donde escribes las órdenes y ves las respuestas	Se instala junto con PostgreSQL
Un editor de texto	Para escribir tu código en archivos	Bloc de notas, o mejor, Visual Studio Code
Una carpeta de prácticas	Donde guardarás tus archivos de código	<code>practicas</code> dentro de tus Documentos

¿Para qué sirve?

Para que no te encuentres a mitad de un ejemplo sin saber dónde escribirlo. Todo el manual funciona con estas piezas, y no necesitas ninguna otra.

¿Por qué lo necesito?

Escribir código es una destreza práctica, como conducir. Quien solo lee y nunca escribe se queda sin la mitad del aprendizaje: los errores, que son los que más enseñan, solo aparecen al probar.

¿Cómo funciona?

Hay **dos programas** que trabajan juntos:

1. El **servidor** (PostgreSQL), que se queda funcionando en segundo plano esperando peticiones. Es el que guarda los datos.
2. El **cliente** (`psql`), que es el que abres tú para enviarle órdenes y leer sus respuestas.

Cuando instalas PostgreSQL, el servidor queda instalado y se arranca por sí solo. El cliente `psql` viene incluido.

Dos conceptos que usarás desde el principio:

- **Terminal** (también llamada consola, línea de comandos o *shell*): una ventana donde, en lugar de hacer clic, escribes órdenes de texto y pulsas Intro. En Windows se llama «Símbolo del sistema», «PowerShell» o «Terminal»; en macOS, «Terminal»; en Linux, «Terminal» o «Consola».

- **Editor de texto:** un programa para escribir texto plano. **Word no sirve** para escribir código, porque guarda formato y cambia las comillas rectas (`'`) por comillas curvas (`'`), y el código deja de funcionar. Usa el Bloc de notas o, mucho mejor, Visual Studio Code (gratuito), que además colorea el código.

Paso a paso: preparar tu espacio de trabajo

1. **Crea una carpeta** llamada `practicas` . En Windows, por ejemplo, dentro de `Documentos` . Aquí guardarás todos tus archivos `.sql` .
2. **Instala un editor de texto** si no tienes uno (Visual Studio Code es una buena opción).
3. **Comprueba que sabes abrir una terminal** (los pasos de apertura están en el punto 0.3).
4. **Sigue con el punto 0.2** para instalar PostgreSQL.

Código y resultado

En este punto aún no se escribe código: es una lista de comprobación.

Ejemplo sencillo

Un alumno con Windows prepara: la carpeta `Documentos\practicas` , el Bloc de notas (que ya viene instalado) y, en el punto siguiente, el instalador de PostgreSQL.

Ejemplo real

En una empresa, cada programador tiene en su ordenador un PostgreSQL propio para probar sin tocar los datos reales. Es exactamente lo que vas a montar.

Profundizando

Practicar sin instalar nada. Más adelante (Parte 10) verás Supabase, que ofrece una base de datos PostgreSQL en internet con un editor de SQL en el navegador. Es una alternativa válida, pero para aprender de verdad conviene tener también tu propia instalación local, que te permite equivocarte sin límite ni consecuencias.

Qué ocupa. PostgreSQL es ligero para uso de aprendizaje. No necesitas un ordenador potente.

Ejercicio

1. Escribe en una frase la diferencia entre el servidor y el cliente.
2. ¿Por qué Word no sirve como editor para escribir SQL?
3. Crea la carpeta `practicas` y comprueba que la encuentras desde tu explorador de archivos.

Solución

1. El servidor (PostgreSQL) guarda los datos y responde; el cliente (`psql`) es el programa con el que tú le envías las órdenes.
2. Porque guarda formato y sustituye las comillas rectas por curvas, y el código con comillas curvas da errores.

3. Respuesta personal: la carpeta debe aparecer en tu explorador.

Error habitual

Escribir el código en un procesador de textos (Word) o copiarlo desde una página web con comillas «bonitas». El gestor responde con errores de sintaxis incomprensibles.

Buena práctica

Escribe siempre el código en un editor de texto plano y guárdalo con extensión `.sql`.

Comprobación

Tienes: una carpeta `practicas`, un editor de texto plano y claro qué es una terminal. Si falta alguno, resuélvelo antes de seguir.

0.2 Instalar PostgreSQL

¿Qué es?

Instalar PostgreSQL es poner en tu ordenador el programa gestor y su cliente `psql`. Hay varios caminos según tu sistema operativo.

¿Para qué sirve?

Para tener tu propio gestor con el que practicar. Todo lo que aprendas funciona igual en tu servidor de producción.

¿Por qué lo necesito?

Sin un gestor instalado, no hay donde ejecutar el código del manual. Hacerlo en local es seguro: si borras algo por error, solo pierdes datos de prueba.

¿Cómo funciona?

Elige **un** camino:

Camino	Cuándo elegirlo
Instalador oficial (Windows)	Usas Windows y quieres lo más sencillo
Postgres.app o Homebrew (macOS)	Usas Mac
Gestor de paquetes (Linux)	Usas Ubuntu, Debian o similar
Docker (cualquiera)	Ya conoces Docker o prefieres no instalar nada «dentro» de tu sistema

En todos los casos, durante la instalación se te pedirá o se definirá una **contraseña** para el usuario administrador, llamado `postgres`. **Anótala**: la necesitarás para conectarte.

Paso a paso

Windows (instalador oficial).

1. Entra en postgresql.org/download, elige «Windows» y descarga el instalador (lo publica la empresa EDB).
2. Ejecútalo. Acepta la carpeta de instalación y los componentes que propone (servidor PostgreSQL, pgAdmin 4 y herramientas de línea de comandos).
3. Cuando pida una **contraseña para el superusuario postgres**, escribe una que recuerdes y anótala.
4. Deja el **puerto** por defecto (5432) y el idioma por defecto.
5. Termina la instalación. Si ofrece una herramienta adicional («Stack Builder»), puedes omitirla.

6. En el menú Inicio, busca «SQL Shell (psql)»: es el cliente (punto 0.3).

macOS.

1. Opción sencilla: descarga **Postgres.app** desde postgresapp.com, muévela a Aplicaciones, ábrela y pulsa «Initialize» para crear un servidor.
2. Opción con Homebrew: `brew install postgresql@16` y después `brew services start postgresql@16`. Al terminar, Homebrew puede indicar que añadas la carpeta `bin` de PostgreSQL a tu `PATH` (la lista de carpetas donde el sistema busca programas); hazlo si te lo pide.

Linux (Ubuntu o Debian). Abre una terminal y escribe:

```
sudo apt update
sudo apt install postgresql
```

El servidor se arranca solo. El usuario administrador se usa desde el usuario del sistema `postgres`: `sudo -u postgres psql`.

Docker.

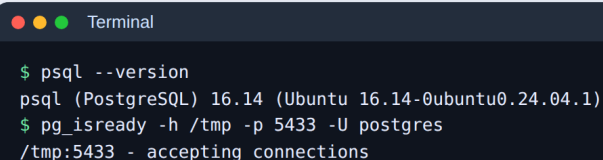
```
docker run --name pg-curso -e POSTGRES_PASSWORD=curso -p 5432:5432 -d
postgres:16
docker exec -it pg-curso psql -U postgres
```

La primera orden arranca un contenedor con PostgreSQL 16 (con la contraseña `curso`, solo para prácticas) y la segunda abre `psql` dentro de él. Si borras el contenedor, pierdes los datos; para conservarlos, añade `-v pgdata:/var/lib/postgresql/data` a la primera orden.

Código y resultado

Para comprobar que la instalación funciona, abre una terminal y escribe estas dos órdenes (la segunda, `pg_isready`, pregunta al servidor si está listo). **En la captura aparecen las opciones `-h /tmp -p 5433 -U postgres` porque el servidor del entorno donde se generó el manual está configurado así; en tu instalación normalmente basta con `pg_isready`.**

```
psql --version
pg_isready -h /tmp -p 5433 -U postgres
```



```
Terminal
$ psql --version
psql (PostgreSQL) 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)
$ pg_isready -h /tmp -p 5433 -U postgres
/tmp:5433 - accepting connections
```

Figura 0.2-a. Captura real: versión del cliente y estado del servidor.

Qué ves en la imagen.

1. `psql --version` muestra la versión del cliente: `psql (PostgreSQL) 16.14 ...`. Si ves un número de versión, el cliente está instalado y la terminal lo encuentra.
2. `pg_isready` responde `accepting connections` («aceptando conexiones»): el servidor está en marcha. En tu instalación aparecerá `localhost:5432 - accepting connections`, con el puerto por defecto.

Ejemplo sencillo

En Ubuntu: `sudo apt install postgresql`, luego `psql --version`. Si muestra una versión, está instalado.

Ejemplo real

En una empresa, el equipo instala la **misma versión mayor** de PostgreSQL en todos los ordenadores y en el servidor, para evitar diferencias de comportamiento entre desarrollo y producción.

Profundizando

Versiones. PostgreSQL publica una versión mayor al año. Lo que aprendes en este manual funciona en la 16 y, salvo detalles muy concretos, en las posteriores. Cuando una función exige una versión mínima, el manual lo indica.

Puerto. El puerto por defecto es el **5432**. Si ya tienes otro PostgreSQL instalado, puede usar otro (por ejemplo el 5433). Si una conexión falla, el puerto es una de las primeras cosas que comprobar.

Servidor y arranque. Normalmente el servidor arranca con el ordenador. En Windows aparece como un «servicio»; en Linux se gestiona con `systemctl`. Lo vemos en la Parte 8.

Ejercicio

1. Elige tu camino de instalación, instala PostgreSQL y anota la contraseña de `postgres`.
2. Abre una terminal y ejecuta `psql --version`.
3. ¿Qué indica `accepting connections`?

Solución

1. Respuesta personal: debes tener instalado el servidor y anotada la contraseña.
2. Debe mostrarse una versión, por ejemplo `psql (PostgreSQL) 16.x`. Si dice que el comando no se encuentra, mira el punto 0.8.
3. Que el servidor está en marcha y listo para recibir conexiones.

Error habitual

Olvidar la contraseña del usuario `postgres` durante la instalación. No se puede recuperar fácilmente: hay que reinstalar o restablecerla con herramientas del administrador. Anótala en cuanto la crees.

Buena práctica

Para prácticas usa una contraseña sencilla que no uses en ningún otro sitio. En producción, jamás (Parte 17).

Comprobación

`psql --version` muestra una versión y `pg_isready` dice que acepta conexiones.

0.3 Abrir `psql` y conectarte

¿Qué es?

`psql` es el programa donde escribes las órdenes para PostgreSQL. Al abrirlo, se conecta al servidor con un usuario y a una base de datos concretos, y te muestra un **prompt**: una línea que indica que está esperando que escribas.

¿Para qué sirve?

Es tu «cuaderno de prácticas»: aquí creas tablas, guardas datos y haces consultas, y ves al instante qué responde el gestor.

¿Por qué lo necesito?

Es la forma más directa, rápida y fiable de hablar con PostgreSQL. Los profesionales lo usan a diario, incluso cuando también usan herramientas gráficas (Parte 7).

¿Cómo funciona?

Para conectarte necesitas cuatro datos, que se indican con **opciones**:

Opción	Significa	Valor típico en tu ordenador
<code>-h</code>	Servidor (<i>host</i>): dónde está	<code>localhost</code> (este ordenador)
<code>-p</code>	Puerto	<code>5432</code>
<code>-U</code>	Usuario	<code>postgres</code>
<code>-d</code>	Base de datos	<code>postgres</code> (la que existe al principio)

Si no indicas una opción, `psql` usa un valor por defecto (`localhost` , `5432` , tu usuario del sistema), así que en una instalación normal basta con:

```
psql -U postgres
```

Y te pedirá la contraseña. **Cuando escribes la contraseña no se ve nada en pantalla: es normal**, escribe y pulsa Intro.

Paso a paso: abrir `psql`

En Windows (la forma más fácil).

1. Abre el menú Inicio y busca «**SQL Shell (psql)**».
2. Se abre una ventana negra que pregunta, una a una, cuatro cosas: `Server [localhost]:` , `Database [postgres]:` , `Port [5432]:` y `Username [postgres]:` . **Pulsa Intro en cada una** para aceptar el valor entre corchetes.

3. Te pide `Password for user postgres:`. Escribe la contraseña que anotaste (no se verá) y pulsa Intro.
4. Si todo va bien, verás `postgres=#`. Ya estás dentro.

En cualquier sistema, desde una terminal.

1. Abre una terminal (Windows: busca «Terminal», «PowerShell» o «Símbolo del sistema»; macOS: busca «Terminal»; Linux: busca «Terminal»).
2. Escribe `psql -U postgres` y pulsa Intro. (En Linux, si se queja del usuario, usa `sudo -u postgres psql`.)
3. Escribe la contraseña si te la pide.
4. Verás el prompt `postgres=#`.

Para **salir**, escribe `\q` y pulsa Intro.

Código y resultado

Esto es lo que ves al abrir `psql` y hacer las primeras comprobaciones. Dentro de la ventana, escribirías solo lo que sigue al prompt `postgres=#`.

```
SELECT 2 + 3 AS suma;
SELECT 'Hola, PostgreSQL' AS saludo;
SELECT current_database() AS base_actual, current_user AS usuario;
```

```
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# SELECT 2 + 3 AS suma;
 suma
-----
    5
(1 row)

postgres=# SELECT 'Hola, PostgreSQL' AS saludo;
 saludo
-----
Hola, PostgreSQL
(1 row)

postgres=# SELECT current_database() AS base_actual, current_user AS usuario;
 base_actual | usuario
-----+-----
 postgres   | postgres
(1 row)

postgres=# \q
```

Figura 0.3-a. Captura real: una sesión interactiva de `psql`, desde que se abre hasta que se sale.

Qué ves en la imagen.

1. Las dos primeras líneas son la **cabecera** que muestra `psql` al conectarse: su versión y la pista `Type "help" for help` («escribe `help` para ayuda»).

2. `postgres=#` es el **prompt**. La parte `postgres` es el nombre de la base de datos a la que estás conectado; el `#` indica que eres administrador (si fueras un usuario normal verías `>`).
3. Después del prompt va lo que escribe quien practica: `SELECT 2 + 3 AS suma;`. La orden `SELECT` pide un resultado; `2 + 3` es una operación; `AS suma` pone nombre `suma` a la columna del resultado; el `;` cierra la orden.
4. `psql` responde con una **tabla de resultados** de una columna (`suma`) y una fila (`5`), y la línea (`1 row`).
5. La segunda orden devuelve un texto, y la tercera pregunta a PostgreSQL en qué base de datos estás y con qué usuario.
6. `\q` cierra `psql`.

Ejemplo sencillo

Abres `psql`, escribes `SELECT 2 + 3 AS suma;`, pulsas Intro y ves `5`. Ya has hablado con PostgreSQL.

Ejemplo real

Un administrador se conecta por la terminal al servidor de la empresa con `psql -h servidor.empresa.com -U administrador -d tienda` para revisar un dato. Es el mismo programa, con otras opciones.

Profundizando

El prompt cambia. `postgres=#` significa «listo para una orden nueva». `postgres-#` significa «la orden anterior no ha terminado» (te falta el `;`). `postgres(#` significa «hay un paréntesis sin cerrar». Lo verás con una captura en el punto 0.4.

Historial. Con las flechas `↑` y `↓` recuperas las órdenes anteriores, para repetirlas o corregirlas, sin volver a escribirlas.

Conectarte a otro sitio. La misma sintaxis (`-h`, `-p`, `-U`, `-d`) sirve para conectarte a un VPS o a una base de datos en la nube; solo cambian los valores (Partes 8 a 10).

Varios usuarios. `postgres` es el administrador. Para aplicaciones se crean usuarios con menos permisos (Parte 6 y 17).

Ejercicio

1. Abre `psql`, conéctate y ejecuta `SELECT 7 * 6 AS respuesta;`. ¿Qué resultado ves?
2. ¿Qué significa el `#` del prompt?
3. Sal de `psql` y vuelve a entrar.

Solución

1. Ves una columna `respuesta` con el valor `42`.
2. Que estás conectado como administrador (superusuario).

3. `\q` para salir; `psql -U postgres` (o «SQL Shell» en Windows) para entrar.

Error habitual

Escribir la contraseña y pensar que no funciona porque «no se ve nada». Es normal: por seguridad, la terminal no muestra lo que escribes. Escribe y pulsa Intro.

Buena práctica

Tras conectarte, comprueba siempre en qué base de datos estás (el nombre antes de `=#`) antes de crear o borrar nada.

Comprobación

Ves `postgres=#`, y `SELECT 2 + 3;` devuelve 5.

0.4 Tu primera orden: cómo se escribe una sentencia

¿Qué es?

Una **sentencia** (o **orden**) es una instrucción SQL completa. Tiene una forma fija: palabras clave en un orden, y un punto y coma final.

¿Para qué sirve?

Para que el gestor sepa **dónde empieza y dónde acaba** lo que le pides, y qué debe hacer.

¿Por qué lo necesito?

Los errores más frecuentes de principiante no son de lógica, sino de escritura: olvidar el `;`, escribir mal una palabra, abrir comillas y no cerrarlas. Conocer las reglas evita horas de frustración.

¿Cómo funciona?

Reglas básicas:

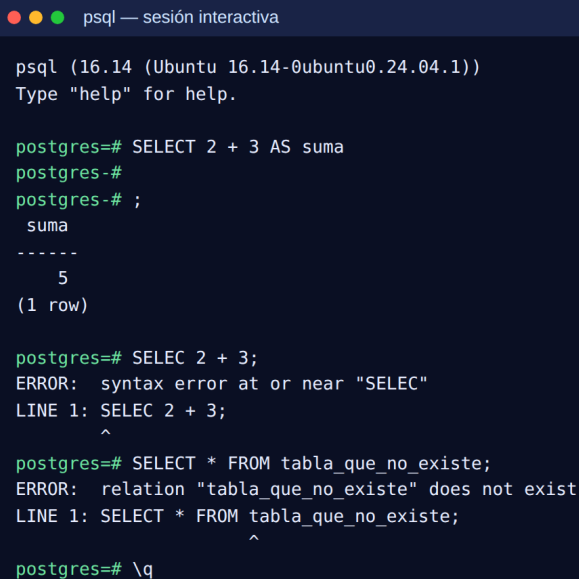
1. **Termina cada sentencia con `;`**. Sin él, `psql` entiende que sigues escribiendo y espera.
2. **Las palabras clave no distinguen mayúsculas de minúsculas:** `SELECT`, `select` y `Select` son lo mismo. En este manual escribimos las palabras clave en MAYÚSCULAS para distinguirlas de tus nombres.
3. **Los textos van entre comillas simples** (`'Hola'`), nunca dobles.
4. **Puedes escribir una sentencia en varias líneas;** da igual dónde pongas los saltos, solo importa el `;` final.
5. **Los comentarios** empiezan con `--` y llegan hasta el final de la línea; el gestor los ignora.
6. Los comandos de `psql` (los que empiezan por `\`) **no llevan `;`**.

Paso a paso: ver qué pasa cuando te equivocas

1. **Escribe una sentencia sin `;`** y pulsa Intro. Verás que el prompt cambia a `postgres-#`.
2. **Escribe `;` en la línea siguiente.** La sentencia se ejecuta.
3. **Escribe una palabra clave con una errata** (`SELEC`) y lee el error.
4. **Consulta una tabla que no existe** y lee el error.
5. Observa que, tras un error, no ha pasado nada grave: puedes seguir.

Código y resultado

```
SELECT 2 + 3 AS suma
;
SELEC 2 + 3;
SELECT * FROM tabla_que_no_existe;
```



```
psql — sesión interactiva

psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# SELECT 2 + 3 AS suma
postgres=#
postgres=# ;
 suma
-----
    5
(1 row)

postgres=# SELEC 2 + 3;
ERROR:  syntax error at or near "SELEC"
LINE 1: SELEC 2 + 3;
        ^

postgres=# SELECT * FROM tabla_que_no_existe;
ERROR:  relation "tabla_que_no_existe" does not exist
LINE 1: SELECT * FROM tabla_que_no_existe;
                        ^

postgres=# \q
```

Figura 0.4-a. Captura real: una sentencia sin punto y coma, y dos errores típicos.

Qué ves en la imagen.

1. `SELECT 2 + 3 AS suma` sin `;`: al pulsar Intro, el prompt cambia a `postgres=#`: `psql` está esperando más. Se pulsa Intro otra vez (línea vacía) y sigue esperando. Al escribir `;` en la línea siguiente, la sentencia se ejecuta y aparece el resultado, `5`. La sentencia ocupó tres líneas y funcionó igual.
2. `SELEC 2 + 3;` produce `ERROR: syntax error at or near "SELEC"` («error de sintaxis cerca de `SELEC`»). `psql` muestra la línea y una flecha `^` justo donde está el problema.
3. `SELECT * FROM tabla_que_no_existe;` produce `ERROR: relation "tabla_que_no_existe" does not exist`. En lenguaje de bases de datos, «relación» significa aquí «tabla»: la tabla no existe.
4. Tras cada error, el prompt vuelve a `postgres=#`: puedes escribir otra orden. **Un error no rompe nada.**

Ejemplo sencillo

`SELECT 'Hola' AS saludo;` devuelve una fila con el texto `Hola` en la columna `saludo`.

Ejemplo real

Un programador escribe una consulta de diez líneas, olvida el `;` y ve el prompt `postgres-#`. Sabe al instante lo que pasa, escribe `;` y continúa.

Profundizando

Cancelar lo que estás escribiendo. Si escribes mal y quieres descartar la sentencia a medias, pulsa **Ctrl + C**: el prompt vuelve a `postgres=#`.

Comentarios. `-- esto es un comentario` se ignora. Sirven para explicar el código, y para «apagar» temporalmente una línea.

Mayúsculas en los datos. Las palabras clave no distinguen mayúsculas, pero los **textos** sí: `'Ana'` y `'ana'` son valores distintos.

Mensajes en inglés. Los mensajes de error están en inglés porque así los emite PostgreSQL. Aprender a leerlos es una destreza clave: casi siempre dicen exactamente qué falla y dónde.

Ejercicio

1. Escribe `SELECT 10 / 4;` y observa el resultado (¿es 2, 2.5 o 2.50?). Después escribe `SELECT 10 / 4.0;` y compara.
2. Escribe una sentencia a propósito con una errata en `FROM` y lee el error.
3. Escribe una sentencia en tres líneas con comentarios.

Solución

1. `10 / 4` da `2` (con enteros, la división descarta los decimales); `10 / 4.0` da `2.5000000000000000`. Cuando uno de los números es decimal, el resultado es decimal.
2. Por ejemplo `SELECT 1 FORM tabla;` da `syntax error at or near "FORM"`.
3. Por ejemplo:

```
SELECT 2 + 3 -- esto suma
AS suma;    -- y ponemos nombre a la columna
```

Error habitual

Olvidar el `;` y creer que `psql` «se ha colgado». No: está esperando. Escribe `;` y Intro.

Buena práctica

Cuando algo no responde, mira el prompt: `postgres-#` significa «falta el `;`». Y lee **siempre el mensaje de error completo**, de arriba abajo.

Comprobación

Sabes ejecutar una sentencia, reconocer el prompt `postgres-#`, cancelar con **Ctrl + C** y distinguir una errata de sintaxis de una tabla inexistente.

0.5 Crear tus bases de datos de práctica

¿Qué es?

Una **base de datos** es un conjunto de tablas independiente de las demás (punto 1.3). Vamos a crear dos para practicar: **ensayo**, tu cuaderno de borrador (puedes borrarla y recrearla cuando quieras), y **tienda**, donde construiremos la tienda del manual.

¿Para qué sirve?

Para no practicar en la base de datos **postgres**, que es la del sistema. Tener tus propias bases te permite equivocarte sin consecuencias y empezar de cero cuando quieras.

¿Por qué lo necesito?

Los ejemplos del manual usan estas bases de datos. Crearlas ahora evita errores de «la base de datos no existe» más adelante.

¿Cómo funciona?

Tres órdenes que usarás mucho:

Orden	Qué hace
<code>CREATE DATABASE nombre;</code>	Crea una base de datos nueva
<code>\l</code>	Lista las bases de datos (la «l» es de <i>list</i>)
<code>\c nombre</code>	Te conecta a esa base de datos (la «c» es de <i>connect</i>)

Para **empezar de cero** una base de datos: `DROP DATABASE ensayo;` la borra y `CREATE DATABASE ensayo;` la crea vacía. **Cuidado:** `DROP` borra para siempre y sin preguntar; en tus prácticas no importa, en datos reales es una de las órdenes más peligrosas (Parte 19).

Paso a paso

1. **Conéctate** con `psql` (punto 0.3). Verás `postgres=#`.
2. **Crea **ensayo****: `CREATE DATABASE ensayo;`. Responde `CREATE DATABASE`.
3. **Crea **tienda****: `CREATE DATABASE tienda;`.
4. **Comprueba que existen** con `\l`.
5. **Conéctate a **ensayo**** con `\c ensayo`. El prompt cambia a `ensayo=#`.
6. **Comprueba que está vacía** con `\dt` (lista las tablas).

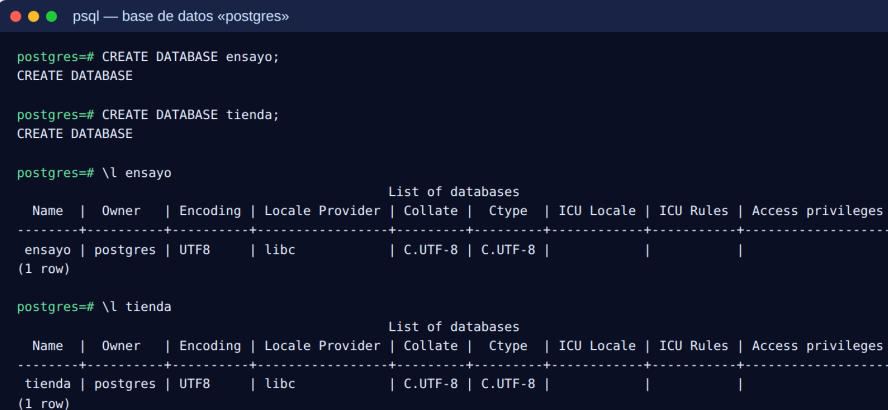
Código y resultado

```
CREATE DATABASE ensayo;

CREATE DATABASE tienda;

\l ensayo

\l tienda
```



```
psql — base de datos «postgres»

postgres=# CREATE DATABASE ensayo;
CREATE DATABASE

postgres=# CREATE DATABASE tienda;
CREATE DATABASE

postgres=# \l ensayo

      List of databases
  Name | Owner  | Encoding | Locale Provider | Collate | Ctype  | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 ensayo | postgres | UTF8     | libc           | C.UTF-8 | C.UTF-8 |             |           | 
(1 row)

postgres=# \l tienda

      List of databases
  Name | Owner  | Encoding | Locale Provider | Collate | Ctype  | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 tienda | postgres | UTF8     | libc           | C.UTF-8 | C.UTF-8 |             |           | 
(1 row)
```

Figura 0.5-a. Captura real: creación de las dos bases de datos y comprobación con `\l`.

Qué ves en la imagen. Cada `CREATE DATABASE` responde `CREATE DATABASE`. `\l ensayo` y `\l tienda` listan solo la base que coincide con el nombre: aparecen con propietario `postgres` y codificación `UTF8` (admite tildes y cualquier alfabeto).

```
\l
\c ensayo
\d
\d
```

```

psql — sesión interactiva

psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \l

               List of databases
  Name | Owner | Encoding | Locale Provider | Collate | Ctype | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 ensayo | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 |  |  | 
 postgres | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 |  |  | 
 template0 | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 |  |  | =c/postgres +
 template1 | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 |  |  | postgres=CTc/postgres +
 tienda | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 |  |  | =c/postgres +
        |         |     |     |         |         |  |  | postgres=CTc/postgres
(5 rows)

postgres=# \c ensayo
You are now connected to database "ensayo" as user "postgres".
ensayo=# \dt
               List of relations
 Schema | Name  | Type | Owner
-----+-----+-----+-----
 public | saludos | table | postgres
(1 row)

ensayo=# \d
               List of relations
 Schema | Name  | Type | Owner
-----+-----+-----+-----
 public | saludos | table | postgres
(1 row)

ensayo=# \q

```

Figura 0.5-b. Captura real: listar bases de datos, cambiar de una a otra con `\c` y ver sus tablas.

Qué ves en la imagen.

1. `\l` lista todas las bases de datos: `ensayo`, `postgres`, `tienda` y dos plantillas (`template0` y `template1`) que PostgreSQL usa por dentro y que no debes tocar.
2. `\c ensayo` responde `You are now connected to database "ensayo" as user "postgres"` («ahora estás conectado a la base `ensayo`»). Fíjate: el prompt pasa de `postgres=#` a `ensayo=#`.
3. `\dt` (tablas de la base actual) y `\d` (todas las relaciones) muestran la tabla `saludos`, que se creó en el punto siguiente.

Ejemplo sencillo

`CREATE DATABASE ensayo;`, `\c ensayo` y ya estás listo para crear tablas en tu cuaderno.

Ejemplo real

En una empresa hay una base de datos `desarrollo` para probar y otra `produccion` con datos reales. Nadie prueba en `produccion`: lo mismo que haces tú con `ensayo`.

Profundizando

Cuántas bases de datos. Puedes crear tantas como quieras; cada una es independiente (las tablas de una no se ven desde otra).

Nombres. Usa minúsculas, sin tildes ni espacios (`tienda`, no `Mi Tienda`).

La base `postgres`. Existe de inicio y sirve para administrar. Evita crear en ella tus tablas.

Plantillas. `CREATE DATABASE` copia en realidad la plantilla `template1`. Por eso no se puede borrar una plantilla ni estar conectado a la base que quieres borrar.

Ejercicio

1. Crea una tercera base de datos llamada `pruebas`, conéctate a ella y comprueba que no tiene tablas.
2. Bórrala con `DROP DATABASE pruebas;` (antes tendrás que salir de ella con `\c postgres`).
3. ¿Cómo sabes en qué base de datos estás?

Solución

1. `CREATE DATABASE pruebas;`, `\c pruebas` y `\dt` responde `Did not find any relations.` («no se ha encontrado ninguna relación»).
2. `\c postgres` y después `DROP DATABASE pruebas;`.
3. Por el prompt (el nombre antes de `=#`) o con `SELECT current_database();`.

Error habitual

Intentar borrar una base de datos estando conectado a ella: PostgreSQL responde que otro usuario (tú mismo) la está usando. Conéctate antes a otra (`\c postgres`).

Buena práctica

Antes de crear tablas, comprueba el prompt: ¿estás en `ensayo`, en `tienda` o en `postgres`? Es el error de principiante más habitual.

Comprobación

`\l` lista `ensayo` y `tienda`, y `\c ensayo` cambia el prompt a `ensayo=#`.

0.6 Escribir código en archivos y ejecutarlo

¿Qué es?

Hasta ahora has escrito órdenes sueltas directamente en `psql`. Lo habitual, y lo que haremos en el manual, es **escribirlas en un archivo de texto con extensión `.sql`** (un **script**) y pedirle a `psql` que lo ejecute entero.

¿Para qué sirve?

Un archivo se puede guardar, corregir, repetir y compartir. Si te equivocas escribiendo directamente en `psql`, debes empezar de nuevo; con un archivo, corriges la línea y vuelves a ejecutarlo.

¿Por qué lo necesito?

Los profesionales no escriben las órdenes importantes «sobre la marcha»: las guardan en archivos. Es también la forma de reconstruir una base de datos desde cero (punto 3.7) y de trabajar en equipo.

¿Cómo funciona?

1. Escribes las sentencias en un archivo, con un editor de texto, y lo guardas con extensión `.sql` en tu carpeta `practicas`.
2. Le dices a `psql` que lo ejecute, de dos formas equivalentes:
 - **Desde dentro de `psql`**, con `\i ruta/del/archivo.sql` (la «i» es de *include*, «incluir»).
 - **Desde la terminal**, con `psql -U postgres -d ensayo -f ruta/del/archivo.sql` (`-f` es de *file*, «archivo»).
1. `psql` ejecuta las sentencias en orden, una por una, y muestra la respuesta de cada una.

Dos detalles importantes:

- **Rutas en Windows.** Dentro de `psql` usa barras normales: `\i C:/Users/TuNombre/Documents/practicas/primer_script.sql`. (Las barras invertidas pueden dar problemas dentro de `psql`.) Si primero entras con `cd` en la carpeta de prácticas desde la terminal, puedes usar solo el nombre del archivo.
- **Ejecutar dos veces.** Si el script crea una tabla y ya existe, dará error la segunda vez. Hay que borrar la tabla antes (`DROP TABLE`) o empezar con una base de datos nueva.

Paso a paso: tu primer script

1. **Abre tu editor** y crea un archivo nuevo.
2. **Escribe** el contenido del código de abajo.
3. **Guárdalo** como `primer_script.sql` en la carpeta `practicas`.
4. **Abre una terminal en esa carpeta** (o usa la ruta completa).

5. **Ejecútalo** contra la base `ensayo`.
6. **Léelo todo**: el script crea una tabla, guarda una fila y la muestra.
7. **Ejecútalo otra vez** y comprueba el error que ocurre.
8. **Borra la tabla y vuelve a ejecutarlo**.

Código y resultado

El archivo `primer_script.sql`:

```
-- Mi primer script: crea una tabla, guarda una fila y la lee
CREATE TABLE saludos (
  id      integer PRIMARY KEY,
  mensaje text NOT NULL
);

INSERT INTO saludos VALUES (1, 'Hola, PostgreSQL');

SELECT * FROM saludos;
```

Y su ejecución desde la terminal con `-f`:

```
cat primer_script.sql
psql -h /tmp -p 5433 -U postgres -d ensayo -f primer_script.sql
```



```
Terminal
$ cat primer_script.sql
-- Mi primer script: crea una tabla, guarda una fila y la lee
CREATE TABLE saludos (
  id      integer PRIMARY KEY,
  mensaje text NOT NULL
);

INSERT INTO saludos VALUES (1, 'Hola, PostgreSQL');

SELECT * FROM saludos;
$ psql -h /tmp -p 5433 -U postgres -d ensayo -f primer_script.sql
CREATE TABLE
INSERT 0 1
 id |      mensaje
-----+-----
  1 | Hola, PostgreSQL
(1 row)
```

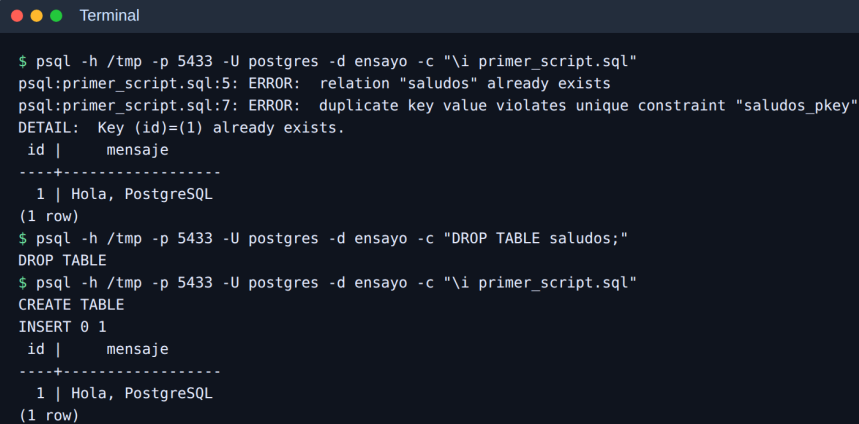
Figura 0.6-a. Captura real: mostrar el archivo y ejecutarlo con ``psql -f``.

Qué ves en la imagen.

1. `cat primer_script.sql` muestra el contenido del archivo (en Windows se usa `type` en lugar de `cat`).
2. `psql ... -d ensayo -f primer_script.sql` ejecuta el archivo contra la base de datos `ensayo`. `psql` imprime la respuesta de cada sentencia, en orden: `CREATE TABLE` (la tabla se ha creado), `INSERT 0 1` (se ha insertado 1 fila) y, por último, el resultado del `SELECT`: una fila con el mensaje.

Ahora ejecutamos el script **desde dentro de psql** con `\i` (aquí con `-c` para verlo en una sola captura), primero cuando la tabla ya existe, después de borrarla:

```
psql -h /tmp -p 5433 -U postgres -d ensayo -c "\i primer_script.sql"
psql -h /tmp -p 5433 -U postgres -d ensayo -c "DROP TABLE saludos;"
psql -h /tmp -p 5433 -U postgres -d ensayo -c "\i primer_script.sql"
```



```
Terminal
$ psql -h /tmp -p 5433 -U postgres -d ensayo -c "\i primer_script.sql"
psql:primer_script.sql:5: ERROR: relation "saludos" already exists
psql:primer_script.sql:7: ERROR: duplicate key value violates unique constraint "saludos_pkey"
DETAIL: Key (id)=(1) already exists.
 id |      mensaje
-----+-----
  1 | Hola, PostgreSQL
(1 row)
$ psql -h /tmp -p 5433 -U postgres -d ensayo -c "DROP TABLE saludos;"
DROP TABLE
$ psql -h /tmp -p 5433 -U postgres -d ensayo -c "\i primer_script.sql"
CREATE TABLE
INSERT 0 1
 id |      mensaje
-----+-----
  1 | Hola, PostgreSQL
(1 row)
```

Figura 0.6-b. Captura real: ejecutar el script otra vez da errores, y se resuelve borrando la tabla.

Qué ves en la imagen.

1. La primera ejecución de `\i primer_script.sql` falla en dos sentencias, y `psql` indica el **archivo y la línea** de cada error: en la línea 5, `relation "saludos" already exists` (la tabla ya existe); en la línea 7, `duplicate key value violates unique constraint "saludos_pkey"` (la fila con `id` 1 ya existe). La última sentencia (`SELECT`) sí se ejecuta y muestra la fila.
2. `DROP TABLE saludos;` borra la tabla (`DROP TABLE`).
3. La segunda ejecución de `\i` funciona sin errores: `CREATE TABLE`, `INSERT 0 1` y el resultado.

Fíjate en que `psql` **continúa tras un error** y ejecuta el resto: por eso hay que leer todos los mensajes, no solo el último.

```
DROP TABLE saludos;
```

```
\dt
```



```
psql — base de datos «ensayo»

ensayo=# DROP TABLE saludos;
DROP TABLE

ensayo=# \dt
Did not find any relations.
```

Figura 0.6-c. Captura real: borrar la tabla y comprobar que la base de datos queda vacía.

Qué ves en la imagen. `DROP TABLE saludos;` responde `DROP TABLE`; `\dt` responde `Did not find any relations.` La base de datos `ensayo` vuelve a estar vacía.

Ejemplo sencillo

Guardar tres sentencias en un archivo y ejecutarlas con `psql -f` equivale a escribirlas una a una, pero repetible.

Ejemplo real

Un proyecto tiene un archivo `crear_tablas.sql` en su repositorio. Cada vez que alguien monta un entorno nuevo, ejecuta ese archivo y obtiene la misma estructura.

Profundizando

Parar al primer error. Con la opción `-v ON_ERROR_STOP=1` (por ejemplo, `psql -v ON_ERROR_STOP=1 -f script.sql`), `psql` se detiene en el primer error en lugar de continuar. Es lo recomendable en scripts de verdad.

IF EXISTS. `DROP TABLE IF EXISTS saludos;` no da error si la tabla no existe, y permite scripts que se pueden ejecutar varias veces. `CREATE TABLE IF NOT EXISTS` hace lo mismo para crear.

Scripts del manual. Cada parte del manual incluye un archivo `.sql` con **todo el código de sus ejemplos**, en orden, para que puedas ejecutarlo sin teclearlo (punto 0.9).

Codificación. Guarda siempre los archivos en **UTF-8** (la opción por defecto de Visual Studio Code) para que las tildes y la ñe se guarden bien.

Ejercicio

1. Crea `mi_tabla.sql` con una tabla `amigos (id integer PRIMARY KEY, nombre text NOT NULL)`, dos filas y un `SELECT`. Ejecútalo.
2. Ejecútalo por segunda vez. ¿Qué errores salen y en qué líneas?
3. Modifica el script para que empiece con `DROP TABLE IF EXISTS amigos;` y ejecútalo varias veces.

Solución

1. Por ejemplo:

```
CREATE TABLE amigos (id integer PRIMARY KEY, nombre text NOT NULL);
INSERT INTO amigos VALUES (1, 'Ana'), (2, 'Luis');
SELECT * FROM amigos;
```

1. En la línea del `CREATE TABLE`: `relation "amigos" already exists`; y en la del `INSERT`: `duplicate key value violates unique constraint "amigos_pkey"`.
2. Con `DROP TABLE IF EXISTS amigos;` como primera línea, se puede ejecutar cuantas veces quieras sin errores.

Error habitual

Guardar el archivo como `primer_script.sql.txt` (el Bloc de notas añade `.txt` si eliges «Todos los archivos» mal) y no encontrarlo después, o ejecutarlo desde otra carpeta y recibir `No such file or directory`.

```
# Mal: psql no encuentra el archivo desde esta carpeta
psql -U postgres -d ensayo -f primer_script.sql # No such file or directory
```

Buena práctica

Guarda todo el código importante en archivos `.sql`, con nombres claros y comentarios (`--`). Empieza los scripts de práctica con `DROP TABLE IF EXISTS` para poder repetirlos.

Comprobación

Sabes crear un `.sql`, ejecutarlo con `\i` y con `-f`, leer los errores con su archivo y línea, y repetirlo tras borrar la tabla.

0.7 Herramientas gráficas (opcional)

¿Qué es?

Además de `psql`, hay programas con ventanas, botones y menús para trabajar con PostgreSQL. Dos de los más conocidos son **pgAdmin** y **DBeaver**. En el manual los veremos con detalle en la Parte 7.

¿Para qué sirve?

Para ver las tablas como cuadrículas, explorar la estructura con el ratón y escribir consultas en un editor con colores. Lo que escribes es el **mismo SQL** que en `psql`.

¿Por qué lo necesito?

No es obligatorio para seguir el manual, pero si prefieres evitar la terminal o ves problemas de tildes en la consola de Windows, es una alternativa cómoda.

¿Cómo funciona?

Todas estas herramientas funcionan igual: te **conectas** a un servidor (indicando servidor, puerto, usuario, contraseña y base de datos), abres un **editor de consultas**, escribes SQL y lo **ejecutas**. Los resultados aparecen en una cuadrícula.

Paso a paso: dónde escribir el código en una herramienta gráfica

1. **pgAdmin** (se instala junto con PostgreSQL en Windows): en el panel izquierdo despliega tu servidor, luego «Databases», haz clic en `ensayo` y abre su **Query Tool** (herramienta de consultas) desde el menú «Tools». Escribe el SQL y ejecútalo con el botón de reproducir o con la tecla F5.
2. **DBeaver**: crea una conexión nueva de tipo PostgreSQL (servidor, puerto, usuario y contraseña), abre un **editor SQL** sobre la base `ensayo`, escribe y ejecuta la sentencia con Ctrl + Intro.
3. **Visual Studio Code** admite extensiones para conectarse a PostgreSQL; se verán en la Parte 7.

Los nombres exactos de menús y botones pueden variar entre versiones. En este manual no hay capturas de estas herramientas porque no se han podido ejecutar en el entorno donde se generaron las capturas.

Código y resultado

No hay código nuevo: el mismo SQL de las capturas se escribe y ejecuta igual en cualquiera de ellas.

Ejemplo sencillo

Abres el Query Tool de pgAdmin sobre `ensayo`, escribes `SELECT 2 + 3 AS suma;`, lo ejecutas y ves una cuadrícula con el 5.

Ejemplo real

Los analistas suelen usar herramientas gráficas para explorar datos, y los desarrolladores usan `psql` y archivos `.sql` para tareas repetibles. Muchos usan ambas.

Profundizando

Qué cambia entre herramientas. Cambia cómo se ven los resultados y cómo se ejecuta cada sentencia; los comandos de `psql` que empiezan por `\` (como `\d` o `\dt`) **solo existen en `psql`**. En una herramienta gráfica, se obtiene lo mismo con el panel de navegación o con consultas al catálogo.

Recomendación. Para este manual conviene aprender primero con `psql` (todas las capturas son de `psql`) y añadir una herramienta gráfica cuando estés cómodo.

Ejercicio

1. Abre una herramienta gráfica (opcional) y ejecuta `SELECT 2 + 3 AS suma;`.
2. ¿Qué comando de `psql` no funcionará en una herramienta gráfica?

Solución

1. Debe aparecer una cuadrícula con una columna `suma` y el valor 5.
2. Los que empiezan por barra invertida (`\l`, `\d`, `\dt`, `\c`...), porque son propios de `psql`.

Error habitual

Escribir `\dt` en el editor de una herramienta gráfica y ver un error de sintaxis: esos comandos son de `psql`, no de SQL.

Buena práctica

Elige una herramienta principal para practicar y mantén todo tu código en archivos `.sql`, que sirven en cualquiera de ellas.

Comprobación

Si usas una herramienta gráfica, comprueba que ejecutar la misma consulta en ella y en `psql` da el mismo resultado.

0.8 Qué hacer cuando algo falla

¿Qué es?

Es una guía de los problemas más comunes al empezar y de cómo se resuelven. Equivocarse es normal y es parte del aprendizaje.

¿Para qué sirve?

Para no quedarte bloqueado con un error que tiene una solución sencilla. La mayoría de los problemas de los principiantes son los mismos diez.

¿Por qué lo necesito?

Un principiante que se atasca con un error de conexión y abandona pierde semanas. Con esta guía lo resuelves en minutos.

¿Cómo funciona?

Casi todos los errores dicen en su propio mensaje qué ocurre. El método general:

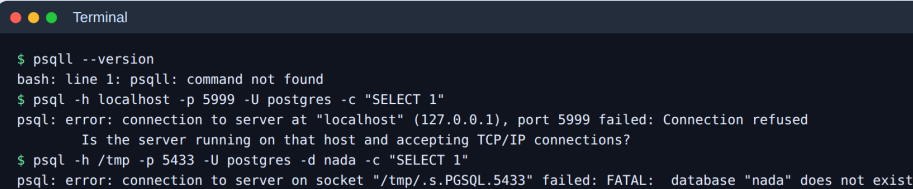
1. **Lee el mensaje completo**, de arriba abajo.
2. **Localiza la palabra clave** del error («does not exist», «refused», «syntax error»).
3. **Mira la línea y la flecha**  que señalan el sitio exacto.
4. **Corrige una cosa cada vez** y vuelve a probar.

Paso a paso: los tres errores más comunes de conexión

Estos tres se reproducen a propósito para que reconozcas su aspecto. Las opciones `-h`, `-p` y `-U` indican el servidor, el puerto y el usuario de este entorno.

Código y resultado

```
psql --version
psql -h localhost -p 5999 -U postgres -c "SELECT 1"
psql -h /tmp -p 5433 -U postgres -d nada -c "SELECT 1"
```



```
Terminal
$ psql --version
bash: line 1: psql: command not found
$ psql -h localhost -p 5999 -U postgres -c "SELECT 1"
psql: error: connection to server at "localhost" (127.0.0.1), port 5999 failed: Connection refused
Is the server running on that host and accepting TCP/IP connections?
$ psql -h /tmp -p 5433 -U postgres -d nada -c "SELECT 1"
psql: error: connection to server on socket "/tmp/.s.PGSQL.5433" failed: FATAL: database "nada" does not exist
```

Figura 0.8-a. Captura real: tres errores típicos del primer día.

Qué ves en la imagen.

1. `psql --version` (con una letra de más): el sistema responde `command not found` («orden no encontrada»). Significa que el nombre del programa está mal escrito, **o que el programa no está en el PATH**. En Windows, el mensaje equivalente es `'psql' no se reconoce como un comando interno o externo`. Solución: revisa la escritura; si es correcta, añade la carpeta `bin` de PostgreSQL al `PATH` o abre «SQL Shell (psql)» desde el menú Inicio.
2. `-p 5999` (un puerto donde no hay servidor): `Connection refused` («conexión rechazada»). El servidor no está en marcha, o el puerto es otro. Solución: comprueba con `pg_isready` que el servidor está activo y usa el puerto correcto (por defecto, 5432).
3. `-d nada` (una base de datos que no existe): `database "nada" does not exist`. Solución: comprueba el nombre con `\l`, o créala con `CREATE DATABASE`.

Otros errores comunes que no se pueden reproducir en el entorno de este manual (porque aquí la conexión local no pide contraseña):

Mensaje	Significa	Solución
<code>password authentication failed for user "postgres"</code>	La contraseña es incorrecta	Escríbela de nuevo; recuerda que no se ve al teclear
<code>permission denied for table ...</code>	Tu usuario no tiene permiso sobre esa tabla	Conéctate como administrador o pide el permiso (punto 1.3; Parte 6)
<code>relation "x" does not exist</code>	La tabla no existe, o estás en otra base de datos	Mira el prompt, usa <code>\dt</code> , y revisa la escritura
<code>syntax error at or near "..."</code>	Hay un fallo de escritura en la palabra señalada	Corrige esa palabra (figura 0.4-a)
El prompt se queda en <code>postgres-#</code>	Falta el <code>;</code>	Escribe <code>;</code> y pulsa Intro

Ejemplo sencillo

Escribes `psql -U postgres` y responde `command not found`: en Linux revisa que el paquete esté instalado; en Windows abre «SQL Shell (psql)».

Ejemplo real

Un compañero llama porque «PostgreSQL no funciona». La primera pregunta es: ¿qué dice exactamente el mensaje? La mayoría de las veces la respuesta ya resuelve el problema.

Profundizando

Buscar el error. Copiar la frase central del mensaje en inglés (por ejemplo, `database "x" does not exist`) en un buscador suele llevar a la solución.

Registros del servidor. Si el servidor no arranca, el motivo suele estar en sus archivos de registro (*logs*); se verán en la Parte 8.

Si te atascas más de media hora. Reduce el problema a la menor sentencia que falla, anótala con su mensaje exacto y pregunta. Una pregunta clara se resuelve rápido.

Ejercicio

1. Provoca a propósito los tres errores de la figura 0.8-a en tu instalación y lee los mensajes.
2. Empareja: `Connection refused` / `database does not exist` / `command not found` con «no está en el PATH», «el servidor no está activo o el puerto es otro» y «esa base de datos no existe».

Solución

1. Cada error debe aparecer con un mensaje equivalente al de la captura (puede variar el sistema operativo).
2. `command not found` con «no está en el PATH»; `Connection refused` con «el servidor no está activo o el puerto es otro»; `database does not exist` con «esa base de datos no existe».

Error habitual

Cambiar varias cosas a la vez al buscar la causa de un error. Después ya no sabes cuál lo arregló, o si has roto algo más.

Buena práctica

Lee primero el mensaje, cambia una cosa cada vez y apunta qué soluciona cada error: acabarás con tu propia guía.

Comprobación

Sabes distinguir un error de conexión (`Connection refused` , `password authentication failed`) de uno de SQL (`syntax error` , `does not exist`) y sabes qué hacer en cada caso.

0.9 Cómo practicar con este manual

¿Qué es?

Es el método de trabajo que seguiremos en todas las partes para que cada ejemplo se convierta en aprendizaje real y no en lectura pasiva.

¿Para qué sirve?

Para que, al terminar cada punto, no solo hayas leído el ejemplo, sino que lo hayas ejecutado, comprendido y puesto a prueba.

¿Por qué lo necesito?

Entender un ejemplo leyéndolo es fácil; reproducirlo y variarlo es lo que fija el conocimiento.

¿Cómo funciona?

Cada punto del manual contiene **código** y la **captura real** de su ejecución. El método:

1. **Lee** la explicación.
2. **Mira** el código y **predice** el resultado antes de ver la captura.
3. **Ejecútalo** tú en `psql` (copia el código a un archivo `.sql`).
4. **Compara** tu resultado con la captura.
5. **Rómpelo a propósito**: cambia un valor, quita una coma, y mira el error.
6. **Haz el ejercicio** sin mirar la solución y compara después.

Paso a paso: un punto de práctica

1. Abre `psql` y conéctate a la base de datos que indique el punto (`tienda` o `ensayo`).
2. Copia el código del ejemplo a un archivo en `practicas` .
3. Ejecútalo con `\i` o `-f` .
4. Compara la salida con la captura. Si difiere, repasa los errores del punto 0.8.
5. Haz el ejercicio y comprueba con la solución.

Código y resultado

Scripts por parte. Para cada parte del manual se entrega un archivo `.sql` (por ejemplo, `parte-01.sql`) con todo el código de las capturas, en el mismo orden y con comentarios que indican la figura y la base de datos de cada bloque. Puedes ejecutarlo entero con `\i` o copiar solo los bloques que quieras practicar. Algunas sentencias de esos archivos **fallan a propósito** (para mostrar un error); los comentarios lo señalan.

Ejemplo sencillo

Punto 1.4: abres `tienda`, ejecutas el script de la tabla `clientes`, comparas con la figura y haces el ejercicio de `productos`.

Ejemplo real

Los programadores aprenden así: leen, prueban, rompen y repiten. Tu `ensayo` es tu campo de pruebas.

Profundizando

Empezar de cero. Si quieres repetir un punto desde cero: `\c postgres`, `DROP DATABASE tienda;` y `CREATE DATABASE tienda;`. Luego vuelve a ejecutar los scripts.

No copies a ciegas. Si copias código que no entiendes, no aprendes. Si no entiendes una línea, vuelve a la explicación línea a línea del punto.

Hacer preguntas. Cuando algo falle, anota: qué quisiste hacer, qué escribiste exactamente y qué mensaje obtuviste. Con esas tres cosas, cualquier persona puede ayudarte.

Ejercicio

1. Ejecuta tu `primer_script.sql` de nuevo en `tienda` y comprueba con `\dt` en qué base de datos se ha creado.
2. Escribe en tus palabras los seis pasos del método de práctica.

Solución

1. Con `psql -U postgres -d tienda -f primer_script.sql`, la tabla `saludos` aparece en `tienda` y no en `ensayo`, porque cada base de datos es independiente.
2. Leer; predecir; ejecutar; comparar; romper a propósito; hacer el ejercicio.

Error habitual

Ejecutar el código en la base de datos equivocada y luego no entender por qué «la tabla no existe» o «ya existe».

Buena práctica

Antes de ejecutar nada, mira el prompt. Y guarda todo tu código en archivos `.sql` con nombres claros.

Comprobación

Tienes: `psql` funcionando, `ensayo` y `tienda` creadas, un `primer_script.sql` ejecutado, y el método de práctica claro.

Resumen de la Parte 0

- Se aprende escribiendo código: necesitas PostgreSQL, `psql`, un editor de texto plano y una carpeta de prácticas.
- El **servidor** guarda los datos y el **cliente** (`psql`) te permite enviarle órdenes.
- Para conectarte indicas servidor, puerto, usuario y base de datos (`-h`, `-p`, `-U`, `-d`); por defecto basta con `psql -U postgres`.
- El prompt `postgres=#` significa «listo»; `postgres-#` significa «falta el `;`».
- Las sentencias terminan en `;`; los comandos de `psql` (`\l`, `\c`, `\dt`, `\d`, `\i`, `\q`) no.
- Los errores no rompen nada, y casi siempre dicen qué falla: léelos completos.
- El código se escribe en archivos `.sql` y se ejecuta con `\i` o `psql -f`.
- Crea `ensayo` y `tienda`, y practica siguiendo el método: leer, predecir, ejecutar, comparar, romper y hacer el ejercicio.

Glosario de la Parte 0

Término	Significado
Terminal	Ventana donde se escriben órdenes de texto
Servidor	Programa (PostgreSQL) que guarda los datos y responde
Cliente	Programa (<code>psql</code>) con el que envías órdenes al servidor
Prompt	Línea que indica que el programa espera una orden
Sentencia	Instrucción SQL completa, terminada en <code>;</code>
Script	Archivo <code>.sql</code> con varias sentencias
Puerto	Número de la «puerta» por la que se entra al servidor (5432 por defecto)
<code>PATH</code>	Lista de carpetas donde el sistema busca programas
Contraseña del superusuario	La del usuario <code>postgres</code> , que se define al instalar
Plantilla	Base de datos interna (<code>template0</code> , <code>template1</code>) que no hay que tocar
<code>\q</code> , <code>\l</code> , <code>\c</code> , <code>\dt</code> , <code>\d</code> , <code>\i</code>	Comandos de <code>psql</code> : salir, listar bases, conectar, listar tablas, describir, ejecutar archivo

Mini examen de la Parte 0

1. ¿Qué diferencia hay entre el servidor y el cliente?

2. ¿Por qué no se escribe SQL en Word?
3. ¿Qué cuatro datos necesitas para conectarte y cuáles son las opciones de `psql` para indicarlos?
4. ¿Qué significa el prompt `postgres-#`?
5. ¿Qué diferencia hay entre `SELECT 1;` y `\l` en cuanto al `;`?
6. ¿Cómo se sale de `psql`?
7. ¿Cómo se ejecuta un archivo `.sql` desde dentro de `psql`? ¿Y desde la terminal?
8. ¿Qué indican `psql:script.sql:5:` y la flecha `^` en un error?
9. ¿Qué significa `Connection refused`?
10. ¿Por qué empezar un script de práctica con `DROP TABLE IF EXISTS`?

Respuestas

1. El servidor guarda los datos y responde; el cliente es el programa con el que le envías órdenes.
2. Porque guarda formato y cambia las comillas rectas por curvas, y el código deja de funcionar.
3. Servidor (`-h`), puerto (`-p`), usuario (`-U`) y base de datos (`-d`).
4. Que la sentencia anterior no ha terminado: falta el `;`.
5. `SELECT 1;` es SQL y termina en `;`; `\l` es un comando de `psql` y no lleva `;`.
6. Con `\q`.
7. Con `\i archivo.sql` dentro de `psql`, o con `psql -U postgres -d base -f archivo.sql` desde la terminal.
8. El archivo y la línea donde ocurrió el error, y el punto exacto de la sentencia donde está el problema.
9. Que no hay un servidor escuchando en ese servidor y puerto: no está en marcha, o el puerto es otro.
10. Para poder ejecutarlo varias veces sin que falle porque la tabla ya existe.

Si has acertado 8 o más, estás listo para la Parte 1. Si no, repasa los puntos de las preguntas falladas.